Nimbo Week 1 - DevOps Technical Report

Executive Summary

This report documents the complete DevOps infrastructure built for the Nimbo Data Ingester service. The infrastructure provides end-to-end automation for building, testing, containerizing, and deploying a Kafka-to-HDFS data ingestion pipeline. All components are containerized, CI/CD automated via Jenkins, and configuration externalized for environment-specific deployments.

---

1. Project Architecture Overview

1.1 System Components

Component	Technology	Purpose
Build Tool	Maven 3.9.x	Dependency management, build lifecycle
Runtime	Java 17 (Eclipse Temurin)	Application runtime
Framework	Spring Boot 3.4.3	Dependency injection, configuration, actuator
Message Broker	Apache Kafka 3.9.0	Data source for Netflow records
Storage	Apache Hadoop HDFS 3.4.1	Distributed file system for Parquet storage
Serialization	Apache Avro 1.11.3	Data serialization/deserialization
Columnar Storage	Apache Parquet 1.15.0	Columnar storage format for analytics
Containerization	Docker	Application packaging
Orchestration	Docker Compose	Local development environment
CI/CD	Jenkins	Automated build, test, deployment pipeline
Code Review	Gerrit	Source control and review

---

2. Project Structure & Build System

2.1 Maven Configuration

The project uses a multi-module structure with a single service (data-ingester) that will later become part of a microservices architecture.

Key Decisions:

Decision	Rationale
`groupId: com.nimbo`	Consistent naming across all future microservices
`artifactId: nimbo`	Single artifact for Week 1; future services will be separate
`Spring Boot Parent 3.4.3`	Provides dependency management and plugin defaults
`Java 17`	Required by project specification
`Packaging: jar`	Executable JAR with embedded dependencies

2.2 Dependency Management

Core Dependencies:

Dependency	Version	Purpose
`spring-boot-starter`	3.4.3	Spring framework core
`spring-boot-starter-actuator`	3.4.3	Health checks, metrics
`spring-boot-starter-validation`	3.4.3	Configuration validation
`kafka-clients`	3.9.0	Kafka consumer/producer API
`spring-kafka`	3.3.3	Spring integration for Kafka
`kafka-avro-serializer`	7.8.0	Avro serialization for Kafka
`avro`	1.11.3	Avro schema and serialization
`hadoop-client`	3.4.1	HDFS client
`parquet-hadoop`	1.15.0	Parquet file writing
`parquet-avro`	1.15.0	Avro-to-Parquet conversion
`lombok`	1.18.36	Boilerplate reduction
`logback-classic`	1.5.16	Logging implementation

Security Upgrades Applied:

Dependency	Old Version	New Version	CVEs Fixed

Dependency	Old Version	New Version	CVEs Fixed
Spring Boot Parent	3.2.4	**3.4.3**	CVE-2025-22235, CVE-2024-38820, CVE-2025-22233
Kafka Clients	3.6.1	**3.9.0**	CVE-2025-27817, CVE-2026-35554, CVE-2025-27818, CVE-2024-31141, CVE-2026-33558
Hadoop Client	3.3.6	**3.4.1**	Multiple security patches
Parquet	1.13.1	**1.15.0**	Security patches
Commons IO	2.15.1	**2.18.0**	Security patches

2.3 Build Plugins

Plugin	Purpose
`maven-compiler-plugin` (3.13.0)	Java 17 compilation
`avro-maven-plugin` (1.11.3)	Generates Java classes from Avro schemas
`spring-boot-maven-plugin`	Creates executable JAR with embedded dependencies
`maven-surefire-plugin` (3.5.2)	Unit test execution (JUnit 5)
`maven-failsafe-plugin` (3.5.2)	Integration test execution
`jacoco-maven-plugin` (0.8.12)	Code coverage reporting

2.4 Build Profiles

Profile	Activation	Purpose
`dev`	Default	Local development with dev configuration
`prod`	Manual	Production build with prod configuration
`skip-tests`	Manual	Fast build skipping tests

---

3. Containerization Strategy

3.1 Multi-Stage Dockerfile

The application uses a multi-stage Dockerfile to minimize image size and attack surface.

Build Stage (Stage 1):

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q -B dependency:go-offline
COPY src ./src
RUN mvn -q -B clean package -DskipTests
```

Runtime Stage (Stage 2):

```
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=build /app/target/nimbo.jar app.jar
RUN chown appuser:appgroup app.jar
USER appuser
```

Image Optimizations:

Optimization	Benefit
Alpine Linux base	~130MB image vs ~350MB standard JRE
Non-root user (`appuser`)	Security best practice
Dependency layer caching	Faster rebuilds (dependencies cached)
Multi-stage build	~180MB final image (vs ~700MB with JDK)

3.2 Build Context Optimization

The `.dockerignore` file excludes:

Excluded Pattern	Reason
`target/` , `*.jar`	Build artifacts not needed in image
`.git/` , `.gitignore`	Version control metadata
`*.md` , `docs/`	Documentation (not needed at runtime)
`.env` , `.env.*`	Secrets must not be baked into image
`src/test/`	Test code not needed at runtime

---

4. Local Development Environment (Docker Compose)

4.1 Service Architecture

Service	Image	Purpose
Zookeeper	`confluentinc/cp-zookeeper:7.5.0`	Kafka coordination
Kafka	`confluentinc/cp-kafka:7.5.0`	Message broker (source of Netflow data)
Namenode	`bde2020/hadoop-namenode:2.0.0`	HDFS metadata server
Datanode	`bde2020/hadoop-datanode:2.0.0`	HDFS storage server
Ingester	Local build	The application (optional in full profile)

4.2 Profiles

Profile	Services Started	Use Case
`infra`	Zookeeper, Kafka, Namenode, Datanode	Development with app run from IDE/JAR
`full`	All services including Ingester	Full containerized stack testing

4.3 Networking

All services communicate via a dedicated Docker network (`nimbo-dev-network`) using service discovery (service names as hostnames).

Service Name	Hostname	Port
Kafka	`kafka`	9092
Namenode	`namenode`	9000 (RPC), 9870 (Web UI)
Datanode	`datanode`	-
Ingester	`ingester`	8080 (Health endpoint)

4.4 Persistent Volumes

Volume	Mount Point	Purpose
`namenode-data`	`/hadoop/dfs/name`	HDFS metadata persistence
`datanode-data`	`/hadoop/dfs/data`	HDFS data persistence

5. Configuration Management

5.1 Configuration Hierarchy

```
application.yml (Base config for all profiles)
    ↓
application-dev.yml (Dev-specific overrides)
    ↓
application-prod.yml (Prod-specific overrides)
    ↓
Environment Variables (Runtime overrides)
```

5.2 Configuration Files

File	Purpose	Key Configs
`application.yml`	Base configuration	Logging, storage.parquet, management, ingester.kafka
`application-dev.yml`	Development profile	Local Kafka (`kafka:9092`), Local HDFS (`namenode:9000`)
`application-prod.yml`	Production profile	Company Kafka (10.10.61.51-54), HA HDFS (team1)

5.3 Ingester Configuration

```yaml
ingester:
  kafka:
    bootstrap-servers: ${KAFKA_BOOTSTRAP_SERVERS:...}
    topic: ${KAFKA_TOPIC:netflow}
    consumer-group: ${KAFKA_CONSUMER_GROUP:netflow-java-consumer8}
    enable-auto-commit: false
    poll-timeout-ms: 1000
    max-poll-records: 500
    session-timeout-ms: 30000
    heartbeat-interval-ms: 3000
    max-poll-interval-ms: 300000
```

5.4 Storage Configuration

```
storage:
  hdfs:
    uri: ${HDFS_URI:hdfs://team1}
    user: ${HDFS_USER:team1}
    base-path: ${HDFS_BASE_PATH:/netflow/parquet}
    nameservice: team1
    nodes:
      team1-node1: ${HDFS_NODE1:10.10.61.51:8020}
      team1-node2: ${HDFS_NODE2:10.10.61.52:8020}
      team1-node3: ${HDFS_NODE3:10.10.61.53:8020}
  parquet:
    compression-codec: ${PARQUET_COMPRESSION:SNAPPY}
    block-size-bytes: ${PARQUET_BLOCK_SIZE:134217728}
    page-size-bytes: ${PARQUET_PAGE_SIZE:1048576}
    enable-dictionary: ${PARQUET_ENABLE_DICTIONARY:true}
```

5.5 Environment Variables (.env.example)

Variable	Purpose
`SPRING_PROFILES_ACTIVE`	Profile selection (dev/prod/test)
`KAFKA_BOOTSTRAP_SERVERS`	Kafka broker addresses
`KAFKA_TOPIC`	Source topic name
`KAFKA_CONSUMER_GROUP`	Consumer group ID
`HDFS_URI`	HDFS Namenode URI
`HDFS_BASE_PATH`	Base path for Parquet files
`HDFS_USER`	HDFS user for authentication
`HDFS_NODE1/2/3`	HA Namenode addresses

---

6. CI/CD Pipeline (Jenkins)

6.1 Pipeline Architecture

```
pipeline {
    agent any
    environment {
```

```
        IMAGE_NAME = 'nimbo'
        IMAGE_TAG = "${env.BUILD_NUMBER}"
        SPRING_PROFILES_ACTIVE = 'prod'
    }
    parameters { ... }
    stages {
        Checkout → Build → Unit Tests → Integration Tests → Build Docker
    }
    post { always { cleanup } success { notification } failure { notifica
}
```

6.2 Pipeline Stages

Stage	What It Does	Tools Used
Checkout	Pulls code from Gerrit	Git SCM
Build	Compiles and packages JAR	Maven (`mvn clean package`)
Unit Tests	Runs JUnit 5 tests	Maven Surefire Plugin
Integration Tests	Runs tests with Testcontainers	Maven Failsafe Plugin
Build Docker Image	Creates container image	Docker build
Deploy	Stops old container, runs new one	Docker run with env vars

6.3 Parameters

Parameter	Type	Default	Purpose
`DEPLOY_ACTION`	Choice	`deploy`	deploy/test-only/build-only
`SKIP_TESTS`	Boolean	`false`	Skip unit tests for fast builds
`SKIP_INTEGRATION_TESTS`	Boolean	`false`	Skip integration tests
`KAFKA_OVERRIDE`	String	`""`	Override Kafka servers (for testing)
`HDFS_OVERRIDE`	String	`""`	Override HDFS URI (for testing)

6.4 Deployment Process

The deploy stage performs the following:

```
# 1. Stop existing container
docker stop nimbo-ingester 2>/dev/null || true
docker rm nimbo-ingester 2>/dev/null || true

# 2. Run new container with environment variables
docker run -d \
    --name nimbo-ingester \
    --restart unless-stopped \
    -e KAFKA_BOOTSTRAP_SERVERS=${kafkaServers} \
    -e HDFS_URI=${hdfsUri} \
    -e KAFKA_TOPIC=${env.KAFKA_TOPIC} \
    -e KAFKA_CONSUMER_GROUP=${env.KAFKA_CONSUMER_GROUP} \
    -e HDFS_BASE_PATH=${env.HDFS_BASE_PATH} \
    -e SPRING_PROFILES_ACTIVE=prod \
    -p 8080:8080 \
    ${IMAGE_NAME}:latest

# 3. Health check
curl -s http://localhost:8080/actuator/health
```

6.5 Artifact Archiving

Artifact	Purpose
`target/*.jar`	Executable JAR for deployment
`integration-logs.txt`	Integration test logs for debugging

---

7. Server Discovery & Infrastructure Mapping

7.1 Kafka Cluster Discovery

Server	Service	Port	Role
10.10.61.51	Kafka Broker	9092	Partition 1 Leader
10.10.61.52	HDFS Namenode	9000, 9870	Partition 2 Leader + Namenode
10.10.61.53	Kafka Broker + Zookeeper	9092, 2181	Replica + Zookeeper
10.10.61.54	Kafka Broker	9092	Partition 0 Leader

7.2 Kafka Topic Configuration

Setting	Value
Topic	`netflow`
Partitions	3
Replication Factor	2
Retention	Changed from 7 days to **30 days**
Consumer Group	`netflow-java-consumer8`

7.3 HDFS HA Configuration

```
hdfs://team1 (Nameservice)
    ├── team1-node1 (10.10.61.51:8020)  ← HA Namenode 1
    ├── team1-node2 (10.10.61.52:8020)  ← HA Namenode 2
    └── team1-node3 (10.10.61.53:8020)  ← HA Namenode 3
```

---

8. Testing Strategy

8.1 Unit Tests

Test Class	What It Tests
`KafkaPropertiesTest`	Configuration validation
`AvroDeserializerTest`	Avro deserialization logic
`DataProcessingServiceTest`	Service orchestration with mocks
`NetflowConsumerTest`	Consumer logic with mocked Kafka

8.2 Integration Tests

Test Class	What It Tests	Technology
`StorageIT`	End-to-end storage flow	Testcontainers (HDFS)

8.3 Test Coverage

Tool	Purpose

Tool	Purpose
JaCoCo	Code coverage reporting
JUnit 5	Test framework
Mockito	Mocking for unit tests
AssertJ	Fluent assertions
Awaitility	Async test waiting

---

9. Security Considerations

9.1 Container Security

Measure	Implementation
Non-root user	`USER appuser` in Dockerfile
Minimal base image	Alpine JRE (reduced attack surface)
`.dockerignore`	Prevents secrets being baked into images
Environment variables	Secrets passed at runtime, not in image

9.2 Vulnerability Management

All dependencies were upgraded to address known CVEs:

Vulnerability	Fix
Spring Boot CVE-2025-22235	Upgraded to 3.4.3
Kafka CVE-2025-27817, CVE-2026-35554	Upgraded to 3.9.0
Spring Context CVEs	Upgraded Spring Boot parent

9.3 Pipeline Security

Measure	Implementation
Jenkins credentials	Stored in Jenkins Credentials Store (not in code)
Docker socket mount	Limited to lab environment
No secrets in Jenkinsfile	All credentials externalized

10. Code Quality & Standards

10.1 Java Code Standards

Standard	Implementation
Java 17 features	Records, var, switch expressions
SLF4J + Logback	Structured logging
Lombok	Boilerplate reduction
`@ConfigurationProperties`	Externalized configuration
`@Validated`	Runtime validation

10.2 Package Structure

```
com.nimbo/
├── ingester/
│   ├── config/          # Configuration properties
│   ├── consumer/        # Kafka consumer logic
│   ├── deserializer/    # Avro deserialization
│   ├── service/         # Business orchestration
│   ├── model/           # Data models
│   └── exception/       # Custom exceptions
└── storage/
    ├── config/          # Storage configuration
    ├── service/         # HDFS/Parquet operations
    ├── batching/        # Batch accumulation
    ├── hdfs/            # HDFS file operations
    ├── naming/          # File naming conventions
    └── exception/       # Storage exceptions
```

11. Deployment Environments

11.1 Local Development

```
# Start infrastructure only
docker compose --profile infra up -d
```

```
# Run app from IDE or JAR
java -jar target/nimbo.jar --spring.profiles.active=dev
```

11.2 CI/CD (Jenkins)

- Runs on company Jenkins server (`darkoob:8080`)
- Triggered by Gerrit `patchset-created` events
- Deploys to production environment

11.3 Production

- Company Kafka: `10.10.61.51:9092,10.10.61.53:9092,10.10.61.54:9092`
- Company HDFS: HA cluster `hdfs://team1`
- Containerized application with `prod` profile

---

12. Monitoring & Observability

12.1 Health Checks

Check	Endpoint	Purpose
Application health	`/actuator/health`	Overall service health
Readyness probe	`/actuator/health/readiness`	Service availability
Liveness probe	`/actuator/health/liveness`	Service liveness

12.2 Metrics

Metric	Purpose
Kafka consumer lag	Monitor data pipeline throughput
Batch accumulation	Monitor flush frequency
HDFS write latency	Monitor storage performance

12.3 Logging

Level	Component
INFO	Application startup, major events
DEBUG	Record processing (dev only)
WARN	Kafka/Hadoop connectivity issues
ERROR	Failures requiring intervention

---

13. Workflow Integration

13.1 Gerrit Workflow

1. Developer pushes to `refs/for/master`
2. Gerrit creates a Change with Patch Set
3. Jenkins automatically builds and tests
4. Jenkins posts `Verified +1/-1` vote
5. Reviewers approve with `Code-Review +2`
6. Change is submitted (merged to master)

13.2 Local Development Workflow

```
# 1. Create feature branch
git checkout -b feature/your-feature-name

# 2. Write code and tests

# 3. Build and test locally
mvn clean package
mvn test
docker compose --profile infra up -d
mvn verify -Pintegration-test

# 4. Test containerized version
docker compose --profile full up -d --build
curl http://localhost:8080/actuator/health

# 5. Commit and push to Gerrit
git add .
git commit -m "feat: description"
git push origin HEAD:refs/for/master
```

```
# 6. Address review feedback
git add .
git commit --amend
git push origin HEAD:refs/for/master

# 7. After merge, sync local
git checkout master
git pull origin master
```

---

14. Key Technical Decisions

Decision	Rationale
Spring Boot instead of manual DI	Standard enterprise framework, reduces boilerplate
Externalized configuration	Environment-specific configs without code changes
Multi-stage Dockerfile	Smaller images, faster deployments
Non-root user in containers	Security best practice
Manual offset commits	Ensures no data loss (at-least-once semantics)
Testcontainers for integration tests	Isolated, reproducible, no production data touched
Gerrit + Jenkins	Code review + CI/CD integration
Multiple Kafka brokers	High availability and fault tolerance
HA HDFS configuration	Resilient storage, no single point of failure

---

15. Architecture Decision Records

ADR-001: Service Naming Convention

Decision: Use `com.nimbo` as group ID and `nimbo-{service-purpose}` as artifact ID.

Rationale: Consistent naming across microservices, easy to identify service responsibilities.

ADR-002: Configuration Management Strategy

Decision: Use Spring Boot `application.yml` with environment variable overrides.

Rationale: Environment-specific configs without code changes, follows Spring Boot best practices.

ADR-003: Docker Image Strategy

Decision: Multi-stage build with Alpine JRE base.

Rationale: Smaller image size, reduced attack surface, faster deployments.

ADR-004: Kafka Offset Management

Decision: Manual offset commits with auto-commit disabled.

Rationale: Zero data loss guarantee - only commit offsets after HDFS write succeeds.

---

16. Technical Debt & Future Improvements

Item	Priority	Description
Kubernetes integration	High	Transition from Docker Compose to K8s for production
Metrics pipeline	Medium	Add Prometheus + Grafana for monitoring
Circuit breakers	Medium	Add resilience patterns for downstream failures
Performance testing	Medium	Load test with production data volumes
Secret management	Low	Implement HashiCorp Vault integration
Log aggregation	Low	Add ELK stack for centralized logging